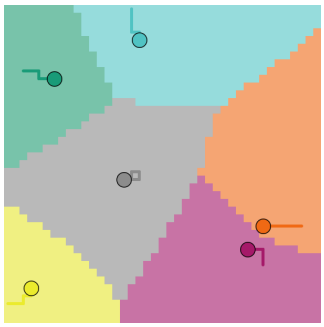
NetLogo Models Library:
Sample Models/Social Science/Economics

(back to the library)

Hotelling's Law



If you download the NetLogo application, this model is included. You can also Try running it in NetLogo Web

WHAT IS IT?

This model is a representation of Hotelling's law (1929), which examines the optimal placement of stores and pricing of their goods in order to maximize profit. In Hotelling's original paper, the stores were confined to a single dimension. This model replicates and extends Hotelling's law, by allowing the stores to move freely on a plane.

In this model, several stores attempt to maximize their profits by moving and changing their prices. Each consumer chooses their store of preference based on the distance to the store and the price of the goods it offers.

HOW IT WORKS

Each consumer adds up the price and distance from each store, and then chooses to go to the store that offers the lowest sum. In the event of a tie, the consumer chooses randomly. The stores can either be constrained to one dimension, in which case all stores operate on a line, or they can be placed on a plane. Under the normal rule, each store tries to move randomly in the four cardinal directions to see if it can gain a larger market share; if not, it does not move. Then each store checks if it can earn a greater profit by increasing or decreasing the price of their goods; if not, it does not change the price. This decision is made without any knowledge of their competitors' strategies. There are two other conditions under which one can run this model: stores can either only change prices, or only move their location.

HOW TO USE IT

Press SETUP to create the stores and a visualization of their starting market share areas. Press GO to have the model run continuously. Press GO-ONCE to have the model run once. The NUMBER-OF-STORES slider decides how many stores are in the world.

If the LAYOUT chooser is on LINE, then the stores will operate only on one dimension. If it is on PLANE, then the stores will operate in a two dimensional space.

If the RULES chooser is on PRICING-ONLY, then stores can only change their price. If it is on MOVING-ONLY, then the stores can only move. If it is on NORMAL, all stores can change their prices and move.

THINGS TO NOTICE

On the default settings, notice that the two stores end up in very close contact and with minimal prices. This is because each store tries to cut into their competitor's fringe consumers by moving closer and reducing their prices.

Also notice how the shapes of the boundaries end up as perpendicular bisectors or hyperbolic arcs. The distance between the stores and their difference in prices determines the eccentricity of these arcs.

Try increasing the store number to three or more, and notice how the store with the most area is not necessarily the most profitable.

Plots show the prices, areas, and revenues of all stores.

THINGS TO TRY

Try to see how stores behave differently when they are either prohibited from moving or changing their prices.

Try increasing the number of stores. Examine how they behave differently and watch which ones are the most successful. What patterns emerge?

EXTENDING THE MODEL

In this model, information is free, but this is not a realistic assumption. Can you think of a way to add a price when stores try to gain information?

In this model, the stores always seek to increase their profits immediately instead of showing any capacity to plan. They are also incapable of predicting what the competitors might do. Making the stores more intelligent would make this model more realistic.

Maybe one way to make the stores more intelligent would be to have them consider their moving and pricing options in conjunction. Right now, they consider the question "would it be good for me to move North" and "would it be good for me to increase my price" completely separately. But what if they asked "would it be good for me to move North AND increase my price"? Would it make a difference in their decision making?

As of now, the consumers are very static. Is there a way to make the consumers move as well or react in some manner to the competition amongst the stores?

Is there a way to include consumer limitations in their spending power or ability to travel? That is, if all stores charge too much or are too far, the consumer could refuse to go to any store.

In this model, if two or more stores are identical from a consumer's point of view, the consumer will choose to go to one of those at random. If the stores are only slightly different for the consumer, is it possible to have the consumer go to either one?

One can extend this model further by introducing a different layout. How would the patterns change, for example, if the layout of the world were circular? What if we just enable world wrapping?

NETLOGO FEATURES

- Each store can move up to four times each tick, as each store takes test steps in cardinal directions when deciding on its moving strategy. However, as this model is tick based, the user only sees one step per store.

- Notice, also, how the plot pens are dynamically created in the setup code of each plot. There has to be one pen for each store, but we don't know in advance how many there will be, so we use the `create-temporary-plot-pen` primitive to create the pens we need and assign them the right color upon setup.

- In the procedures where a store chooses a new price or location, we make use of a subtle property of the `sort-by` primitive: the fact that the order of items in the initial list is preserved in case of ties during sorting. What we do is that we put the "status quo" option at the front of the list possible moves, but shuffle all other possible moves. Because of that, when we sort the moves by potential revenues, the status quo is always preferred in case of equal revenues.

- What if want to know if all members of a list have a certain property? (This is the equivalent of the "for all" universal quantifier (∀) in predicate logic.) NetLogo doesn't have a primitive to do that directly, but we can easily write it ourselves using the `member?` and `map` primitives. The trick is to first map the list to the predicate we want to test. Let's say we want to test if all members of a list are zeros: `map [? = 0] [1 0 0]` will report `[false true true]`, and `map [? = 0] [ 0 0 0 ]` will report `[true true true]`. All we have to is to test the "for all" condition is to make sure that `false` is not a member of that new list: `not member? false map [? = 0] [1 0 0]` will report `false`, and `not member? false map [? = 0] [0 0 0]` will report `true`. We use a variant of this in the `new-price-task` reporter.

- The procedures for choosing new prices and locations do not actually perform these changes right away. Instead, they report `task`s that will be run later in the `go` procedure. Notice how we put `self` in a local variable before creating our task: this is because NetLogo tasks "capture" local variables, but not agent context. See the "Tasks" section in the NetLogo Programming guide for more details about this.

RELATED MODELS

- Voronoi
- Voronoi - Emergent

CREDITS AND REFERENCES

Hotelling, Harold. (1929). "Stability in Competition." The Economic Journal 39.153: 41 -57. (Stable URL: https://www.jstor.org/stable/2224214).

HOW TO CITE

If you mention this model or the NetLogo software in a publication, we ask that you include the citations below.

For the model itself:

- Ottino, B., Stonedahl, F. and Wilensky, U. (2009). NetLogo Hotelling's Law model. http://ccl.northwestern.edu/netlogo/models/Hotelling'sLaw. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL.

Please cite the NetLogo software as:

- Wilensky, U. (1999). NetLogo. http://ccl.northwestern.edu/netlogo/. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL.

COPYRIGHT AND LICENSE

(back to the NetLogo Models Library)